

JAVA LOGIC BUILDING

30 Problems • 3 Difficulty Levels • 7-Step Framework

Every problem solved step by step: Understand → Hand Trace → Pattern → Pseudocode → Java Code → Edge Cases → Optimize

10

BASIC

Problems

10

INTERMEDIATE

Problems

10

HARD

Problems

STEP 1 READ

STEP 2 TRACE

STEP 3
PATTERN

STEP 4
PSEUDOCODE

STEP 5 CODE

STEP 6 TEST

STEP 7
OPTIMIZE

THE 7-STEP FRAMEWORK | Quick Reference Card

STEP	NAME	WHAT TO DO	KEY QUESTION
1	Read & Understand	Read twice. List inputs, outputs, constraints.	What goes in? What comes out?
2	Work by Hand	Solve 2-3 small examples manually on paper.	Can I trace this with real numbers?
3	Identify Pattern	Map to: loop/search/two-ptr/HashMap/stack/DP/graph	Have I seen this before?
4	Pseudocode	Write plain English steps before any Java syntax.	Can I explain this to a non-coder?
5	Write Java Code	Translate pseudocode line-by-line. Get it working first.	Does it compile and produce correct output?
6	Test Edge Cases	Empty input, nulls, negatives, size=1, duplicates.	What could break this?
7	Reflect & Optimize	Reduce time/space complexity. Name the pattern learned.	Can I do better? What did I learn?

PATTERN RECOGNITION CHEAT SHEET

PATTERN	JAVA CLASS/METHOD	RECOGNIZE BY
Counting/Accumulator	int sum=0; for loop	'total', 'sum', 'count'
Two Pointers	int l=0, r=n-1	'pair', 'palindrome', 'sorted', 'reverse'
Sliding Window	int l=0; for r	'substring', 'subarray', 'window'
HashMap Lookup	HashMap<K,V>	'frequency', 'duplicate', 'complement'
Stack	Stack<T> or Deque	'brackets', 'nested', 'LIFO', 'history'
Binary Search	lo/hi/mid	'sorted array', 'O(log n)', 'find target'
DFS/BFS	Stack/Queue + visited	'graph', 'grid', 'tree', 'connected'
Dynamic Programming	int[] dp	'optimal', 'count ways', 'minimum/maximum'
Topological Sort	Queue + inDegree[]	'ordering', 'dependencies', 'DAG'

★ BASIC PROBLEMS (Problems 1–10)

Foundation problems. Master these before moving on. Focus on loops, conditions, arrays, and simple math.

1

Print Numbers 1 to N

BASIC

Given a number N, print all integers from 1 to N, each on a new line.

Pattern

Loop / Counting

Input

N = 5

Output

1 2 3 4 5

Complexity

Time: O(n) | Space: O(1)

1

Read & Understand

Input is an integer N. Output is N lines each with a number. No special constraints.

2

Work by Hand

N=3 → print 1, then 2, then 3. Simple sequential counting.

3

Identify Pattern

This is a basic counting loop. Start at 1, increment by 1, stop at N.

4

Pseudocode

Pseudocode

```
SET i = 1
WHILE i <= N:
    PRINT i
    INCREMENT i
```

5

Write Java Code

Use a for loop from 1 to N inclusive.

Java — Complete Solution

```
public class Main {
    public static void printNumbers(int n) {
        for (int i = 1; i <= n; i++) {
            System.out.println(i);
        }
    }
    public static void main(String[] args) {
        printNumbers(5); // prints 1 2 3 4 5
        printNumbers(0); // prints nothing
    }
}
```

6

Test Edge Cases

Edge: N=0 → print nothing. N=1 → print just 1. N=negative → skip.

7 Reflect & Optimize

Simplest loop. No optimization needed. $O(n)$ time, $O(1)$ space.

Given an integer, determine whether it is even or odd.

Pattern	Input	Output	Complexity
Condition / Modulo	num = 7	Odd	Time: O(1) Space: O(1)

1 Read & Understand

Input: any integer. Output: 'Even' or 'Odd'. Works for negative numbers too.

2 Work by Hand

$7 \% 2 = 1 \rightarrow \text{Odd}$. $4 \% 2 = 0 \rightarrow \text{Even}$. $0 \% 2 = 0 \rightarrow \text{Even}$.

3 Identify Pattern

Single condition check using modulo operator %.

4 Pseudocode

Pseudocode

```
IF num % 2 == 0: PRINT Even
ELSE: PRINT Odd
```

5 Write Java Code

Use if-else with modulo. Works for negatives because Java % preserves sign.

Java — Complete Solution

```
public class Main {
    public static String evenOrOdd(int num) {
        if (num % 2 == 0) {
            return "Even";
        } else {
            return "Odd";
        }
    }
    public static void main(String[] args) {
        System.out.println(evenOrOdd(7)); // Odd
        System.out.println(evenOrOdd(-4)); // Even
        System.out.println(evenOrOdd(0)); // Even
    }
}
```

6 Test Edge Cases

Test: 0 (even), -3 (odd), -4 (even), Integer.MAX_VALUE.

7 Reflect & Optimize

Cannot be simplified further. Modulo is O(1).

3

Sum of Digits of a Number

BASIC

Given a positive integer, return the sum of all its digits.

Pattern

Digit Extraction with % and /

Input

num = 1234

Output

10 (1+2+3+4)

Complexity

Time: $O(d)$ where d = digits |Space: $O(1)$

1 Read & Understand

Input: positive integer. Output: integer (sum of digits). No decimals.

2 Work by Hand

1234: last digit = $1234 \% 10 = 4$, rest = 123. $123 \% 10 = 3$, rest = 12. $12 \% 10 = 2$, rest = 1. $1 \% 10 = 1$. Sum = $4 + 3 + 2 + 1 = 10$.

3 Identify Pattern

Digit extraction: repeatedly take last digit using %, remove it using /10.

4 Pseudocode

Pseudocode

```
SET sum = 0
WHILE num > 0:
    sum += num % 10
    num = num / 10
RETURN sum
```

5 Write Java Code

Loop until num becomes 0. Each iteration extracts the last digit.

Java — Complete Solution

```
public class Main {
    public static int sumOfDigits(int num) {
        int sum = 0;
        while (num > 0) {
            sum += num % 10; // extract last digit
            num /= 10;       // remove last digit
        }
        return sum;
    }
    public static void main(String[] args) {
        System.out.println(sumOfDigits(1234)); // 10
        System.out.println(sumOfDigits(999));  // 27
        System.out.println(sumOfDigits(0));    // 0
    }
}
```

6 Test Edge Cases

Test: $0 \rightarrow 0$. Single digit: $5 \rightarrow 5$. Large: $999 \rightarrow 27$.

7 Reflect & Optimize

Alternative: convert to String and sum chars. Same complexity, but less elegant.

4

Check Prime Number

BASIC

Given a number N, return true if it is prime, false otherwise.

Pattern

Loop + Early Exit
Optimization

Input

N = 17

Output

true

Complexity

Time: $O(\sqrt{n})$ | Space:
 $O(1)$

1 Read & Understand

A prime is divisible only by 1 and itself. Input: integer. Edge cases: $N < 2$ is not prime.

2 Work by Hand

17: check 2,3,4 ($\sqrt{17} \sim 4.1$). None divide evenly $\rightarrow$ prime. 12: 2 divides $\rightarrow$ not prime.

3 Identify Pattern

Only check divisors up to $\sqrt{N}$. If any divide evenly, not prime.

4 Pseudocode

Pseudocode

```
IF n < 2: RETURN false
FOR i = 2 to sqrt(n):
    IF n % i == 0: RETURN false
RETURN true
```

5 Write Java Code

Use `Math.sqrt()` for the upper bound. Cast to `int`.

Java — Complete Solution

```
public class Main {
    public static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false; // divisible = not prime
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPrime(17)); // true
        System.out.println(isPrime(1));  // false
        System.out.println(isPrime(2));  // true
        System.out.println(isPrime(100)); // false
    }
}
```

6 Test Edge Cases

Test: 2 (prime), 1 (not), 0 (not), 4 (not), 13 (prime), 100 (not).

7 Reflect & Optimize

Checking only to $\text{sqrt}(n)$ is the key optimization — reduces $O(n)$ to $O(\text{sqrt}(n))$.

5

Factorial of a Number

BASIC

Given N, return N! (N factorial). Solve iteratively.

Pattern

Accumulator / Iterative
Multiplication

Input

N = 5

Output

120 (5x4x3x2x1)

Complexity

Time: O(n) | Space: O(1)

1 Read & Understand

Factorial: multiply all integers from 1 to N. $0! = 1$ by definition.

2 Work by Hand

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. Build up: $1 \rightarrow 1$, $1 \times 2 = 2$, $2 \times 3 = 6$, $6 \times 4 = 24$, $24 \times 5 = 120$.

3 Identify Pattern

Accumulator pattern. Multiply a running product by each number.

4 Pseudocode

Pseudocode

```
SET result = 1
FOR i = 1 to N:
    result *= i
RETURN result
```

5 Write Java Code

Use long instead of int to handle larger factorials (int overflows at 13!).

Java — Complete Solution

```
public class Main {
    public static long factorial(int n) {
        if (n < 0) throw new IllegalArgumentException("N must be >= 0");
        long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }
    public static void main(String[] args) {
        System.out.println(factorial(0)); // 1
        System.out.println(factorial(5)); // 120
        System.out.println(factorial(10)); // 3628800
    }
}
```

6 Test Edge Cases

Test: $0 \rightarrow 1$, $1 \rightarrow 1$, $5 \rightarrow 120$, $10 \rightarrow 3628800$. For $N > 20$, use BigInteger.

7 Reflect & Optimize

Recursive version: cleaner code but $O(n)$ stack space. Iterative is better for large N.

Given a string, return it reversed without using `StringBuilder.reverse()`.

Pattern	Input	Output	Complexity
Two Pointers	"hello"	"olleh"	Time: $O(n)$ Space: $O(n)$

1 Read & Understand

Input: String. Output: reversed String. Must build manually.

2 Work by Hand

"hello": h<->o swap, e<->l swap, middle l stays. Result: "olleh".

3 Identify Pattern

Two pointers from both ends, swap characters, move inward.

4 Pseudocode

Pseudocode

```
CONVERT string to char array
SET left=0, right=len-1
WHILE left < right:
    SWAP arr[left] and arr[right]
    left++, right--
RETURN new String(arr)
```

5 Write Java Code

Java strings are immutable, so convert to `char[]` first.

Java — Complete Solution

```
public class Main {
    public static String reverseString(String s) {
        char[] arr = s.toCharArray();
        int left = 0, right = arr.length - 1;
        while (left < right) {
            char temp = arr[left]; // swap
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
        return new String(arr);
    }
    public static void main(String[] args) {
        System.out.println(reverseString("hello")); // olleh
        System.out.println(reverseString("a"));     // a
        System.out.println(reverseString(""));      // (empty)
    }
}
```

6 Test Edge Cases

Test: empty string, single char, palindrome, even/odd length.

7 Reflect & Optimize

StringBuilder approach is simpler but the two-pointer shows fundamental logic.

7

Count Vowels in a String

BASIC

Given a string, count the number of vowels (a, e, i, o, u — both cases).

Pattern	Input	Output	Complexity
Linear Scan + Condition	"Hello World"	3	Time: $O(n)$ Space: $O(1)$

1 Read & Understand

Input: String. Output: count of vowel characters. Case-insensitive.

2 Work by Hand

"Hello World": H-no, e-YES, l-no, l-no, o-YES, W-no, o-YES, r-no, l-no, d-no = 3.

3 Identify Pattern

Walk each character. Check if it is a vowel using `String.contains()` or `switch`.

4 Pseudocode

Pseudocode

```
SET count = 0
FOR each char c in string:
    IF toLower(c) is in 'aeiou':
        count++
RETURN count
```

5 Write Java Code

Convert char to lowercase before checking to handle both cases.

Java — Complete Solution

```
public class Main {
    public static int countVowels(String s) {
        int count = 0;
        String vowels = "aeiou";
        for (char c : s.toCharArray()) {
            if (vowels.indexOf(Character.toLowerCase(c)) != -1) {
                count++;
            }
        }
        return count;
    }
    public static void main(String[] args) {
        System.out.println(countVowels("Hello World")); // 3
        System.out.println(countVowels("aeiou"));        // 5
        System.out.println(countVowels("bcd fg"));        // 0
    }
}
```

6 Test Edge Cases

Test: all consonants → 0. All vowels: 'aeiou' → 5. Empty string → 0.

7 Reflect & Optimize

Using a String 'aeiou'.indexOf(c) is concise. Switch-case is fastest but verbose.

Given two integers, find their Greatest Common Divisor (GCD).

Pattern
Euclidean Algorithm

Input
a=48, b=18

Output
6

Complexity
Time: $O(\log(\min(a,b)))$ |
Space: $O(1)$

1 Read & Understand

GCD = largest number that divides both a and b evenly.

2 Work by Hand

GCD(48,18): $48\%18=12 \rightarrow \text{GCD}(18,12)$. $18\%12=6 \rightarrow \text{GCD}(12,6)$. $12\%6=0 \rightarrow \text{GCD is } 6$.

3 Identify Pattern

Euclidean algorithm: $\text{GCD}(a,b) = \text{GCD}(b, a\%b)$ until b becomes 0.

4 Pseudocode

Pseudocode

```
WHILE b != 0:
    SET temp = b
    b = a % b
    a = temp
RETURN a
```

5 Write Java Code

Classic iterative Euclidean algorithm. Very efficient.

Java — Complete Solution

```
public class Main {
    public static int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b; // Euclidean step
            a = temp;
        }
        return a;
    }
    public static void main(String[] args) {
        System.out.println(gcd(48, 18)); // 6
        System.out.println(gcd(7, 7));   // 7
        System.out.println(gcd(0, 5));   // 5
    }
}
```

6 Test Edge Cases

Test: $\text{GCD}(0,5)=5$, $\text{GCD}(7,7)=7$, $\text{GCD}(1,100)=1$, $\text{GCD}(100,0)=100$.

7 Reflect & Optimize

Recursive: $\text{gcd}(a,b) = b==0 ? a : \text{gcd}(b,a\%b)$. Elegant but uses stack. Iterative is safer.

Search for a target value in an unsorted array. Return its index, or -1 if not found.

Pattern
Linear Search

Input
arr={4,2,7,1,9}, target=7

Output
2

Complexity
Time: O(n) | Space: O(1)

1 Read & Understand

Input: int array + target. Output: index of target, or -1. Array may be unsorted.

2 Work by Hand

4≠7, 2≠7, 7==7 → return index 2. Done.

3 Identify Pattern

Walk array one by one. Return index on match. If loop ends without match, return -1.

4 Pseudocode

Pseudocode

```
FOR i = 0 to arr.length-1:
    IF arr[i] == target: RETURN i
RETURN -1
```

5 Write Java Code

Simple for loop. Enhanced for-loop won't work since we need the index.

Java — Complete Solution

```
public class Main {
    public static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i; // found!
            }
        }
        return -1; // not found
    }

    public static void main(String[] args) {
        int[] arr = {4, 2, 7, 1, 9};
        System.out.println(linearSearch(arr, 7)); // 2
        System.out.println(linearSearch(arr, 5)); // -1
    }
}
```

6 Test Edge Cases

Test: target not in array → -1. Target at index 0. Target at last index. Duplicates.

7 Reflect & Optimize

If array were sorted, use Binary Search for O(log n). Linear search is baseline.

Print the multiplication table of a given number N up to 10.

Pattern	Input	Output	Complexity
Loop / Formatted Output	N = 5	5x1=5 5x2=10 ... 5x10=50	Time: O(1) Space: O(1)

1 Read & Understand

Input: integer N. Output: 10 lines showing N*1 through N*10.

2 Work by Hand

5x1=5, 5x2=10, 5x3=15, ... 5x10=50. Each line: N + 'x' + i + '=' + N*i.

3 Identify Pattern

Simple for loop from 1 to 10.

4 Pseudocode

Pseudocode

```
FOR i = 1 to 10:
    PRINT N + 'x' + i + '=' + (N*i)
```

5 Write Java Code

Use String.format() or printf for neat alignment.

Java — Complete Solution

```
public class Main {
    public static void multiTable(int n) {
        for (int i = 1; i <= 10; i++) {
            System.out.printf("%d x %2d = %3d%n", n, i, n * i);
        }
    }
    public static void main(String[] args) {
        multiTable(5);
        // 5 x 1 = 5
        // 5 x 2 = 10
        // ... 5 x 10 = 50
    }
}
```

6 Test Edge Cases

Test: N=0 → all zeros. N=1 → same as i. N=negative → negative products.

7 Reflect & Optimize

printf with formatting: System.out.printf('%d x %2d = %3d%n', n, i, n*i).

★★ INTERMEDIATE PROBLEMS (Problems 11–20)

Core data structure and algorithm problems. HashMap, Stack, Two Pointers, Binary Search, Linked Lists.

11

Two Sum — Find Pair with Target

INTERMEDIATE

Given an integer array and a target, return indices of the two numbers that add up to target.

Pattern

HashMap for $O(n)$ lookup

Input

nums=[2,7,11,15], target=9

Output

[0, 1]

Complexity

Time: $O(n)$ | Space: $O(n)$

1

Read & Understand

Input: int array + target. Output: two indices. Exactly one solution guaranteed. Same element cannot be used twice.

2

Work by Hand

For each element, check if (target - element) was seen before. $9-2=7$ not seen. $9-7=2 \rightarrow$ yes, at index 0!

3

Identify Pattern

Store each visited number and its index in a HashMap. For each new number, look up its complement.

4

Pseudocode

Pseudocode

```
CREATE HashMap seen (value->index)
FOR i = 0 to n-1:
    comp = target - nums[i]
    IF comp in seen: RETURN [seen[comp], i]
    seen.put(nums[i], i)
```

5

Write Java Code

HashMap.containsKey() is $O(1)$. Total is $O(n)$ versus $O(n^2)$ for brute force.

```
import java.util.HashMap;
import java.util.Arrays;
public class Main {
    public static int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> seen = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int comp = target - nums[i];
            if (seen.containsKey(comp)) {
                return new int[]{seen.get(comp), i};
            }
            seen.put(nums[i], i);
        }
        return new int[]{};
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(twoSum(new int[]{2,7,11,15}, 9)));
        // [0, 1]
        System.out.println(Arrays.toString(twoSum(new int[]{3,3}, 6)));
        // [0, 1]
    }
}
```

6 Test Edge Cases

Test: [3,3] target=6 → [0,1]. Negative numbers. Target at start.

7 Reflect & Optimize

Brute force: two nested loops $O(n^2)$. HashMap approach: $O(n)$ time. Always prefer HashMap for pair-search.

Given a string of brackets, return true if every opening bracket has a matching closing bracket in correct order.

Pattern	Input	Output	Complexity
Stack (LIFO)	"({[]})"	true	Time: $O(n)$ Space: $O(n)$

1 Read & Understand

Input: String with `(){}[]`. Output: boolean. Check nesting and matching.

2 Work by Hand

'({[]})': push (, push {, push [. See] → pop [matches. See } → pop { matches. See) → pop (matches. Stack empty → true.

3 Identify Pattern

Stack: push opening brackets. On closing bracket, pop and verify match. If mismatch or empty stack → false.

4 Pseudocode

Pseudocode

```
CREATE Stack
FOR each char:
    IF opening: push
    IF closing: if stack empty OR top != match: return false, else pop
RETURN stack.isEmpty()
```

5 Write Java Code

Use a HashMap to map closing→opening for clean matching logic.

```
import java.util.Stack;
import java.util.HashMap;
public class Main {
    public static boolean isBalanced(String s) {
        Stack<Character> stack = new Stack<>();
        HashMap<Character, Character> match = new HashMap<>();
        match.put(')', '(');
        match.put('}', '{');
        match.put(']', '[');
        for (char c : s.toCharArray()) {
            if ("[{[".indexOf(c) != -1) {
                stack.push(c);
            } else if (match.containsKey(c)) {
                if (stack.isEmpty() || stack.pop() != match.get(c))
                    return false;
            }
        }
        return stack.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(isBalanced("({[]})")); // true
        System.out.println(isBalanced("([)]")); // false
        System.out.println(isBalanced("")); // true
    }
}
```

6 Test Edge Cases

Test: empty → true. Just '(' → false. '()' → false (wrong order).

7 Reflect & Optimize

Classic stack problem. Memorize this pattern — it appears in many variants.

Given an integer array, return a list of all elements that appear more than once.

Pattern	Input	Output	Complexity
Frequency Map (HashMap)	nums={4,3,2,7,8,2,3,1}	[2, 3]	Time: O(n) Space: O(n)

1 Read & Understand

Input: int array. Output: list of duplicates. An element may appear many times but should appear once in result.

2 Work by Hand

Count each element: 4→1, 3→2, 2→2, 7→1, 8→1, 1→1. Those with count>1: 3, 2.

3 Identify Pattern

First pass: count frequencies in HashMap. Second pass: collect elements with count > 1.

4 Pseudocode

Pseudocode

```
CREATE HashMap<Integer,Integer> freq
FOR each num: freq[num]++
CREATE result list
FOR each entry in freq:
    IF value > 1: add key to result
RETURN result
```

5 Write Java Code

Use getOrDefault for clean counting. Return as ArrayList.

Java — Complete Solution

```
import java.util.*;
public class Main {
    public static List<Integer> findDuplicates(int[] nums) {
        HashMap<Integer, Integer> freq = new HashMap<>();
        for (int num : nums) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }
        List<Integer> result = new ArrayList<>();
        for (Map.Entry<Integer,Integer> e : freq.entrySet()) {
            if (e.getValue() > 1) result.add(e.getKey());
        }
        return result;
    }
    public static void main(String[] args) {
        System.out.println(findDuplicates(new int[]{4,3,2,7,8,2,3,1}));
        // [2, 3]
    }
}
```

6 Test Edge Cases

Test: no duplicates → empty list. All same → [that element]. Three of same element.

7 Reflect & Optimize

$O(n)$ space for HashMap. In-place $O(1)$ space trick exists for arrays with values $1..n$: negate visited index. Advanced technique.

Given the head of a singly linked list, reverse it and return the new head.

Pattern

Three-Pointer Reversal
(Iterative)

Input

1 → 2 → 3 → 4 → 5

Output

5 → 4 → 3 → 2 → 1

Complexity

Time: O(n) | Space: O(1)

1 Read & Understand

Input: head of singly linked list. Output: new head after reversal. In-place reversal required.

2 Work by Hand

1→2→3→4→5: At each step, reverse the arrow. 1←2←3←4←5. Need to track previous, current, next.

3 Identify Pattern

Three pointers: prev=null, curr=head, next=null. Reverse curr.next to prev, then advance all three.

4 Pseudocode

Pseudocode

```
SET prev=null, curr=head
WHILE curr != null:
    next = curr.next
    curr.next = prev
    prev = curr
    curr = next
RETURN prev
```

5 Write Java Code

prev becomes the new head when curr is null. Classic pointer manipulation.

```

public class Main {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }
    public static Node reverse(Node head) {
        Node prev = null, curr = head;
        while (curr != null) {
            Node next = curr.next; // save next
            curr.next = prev;      // reverse arrow
            prev = curr;           // move prev forward
            curr = next;           // move curr forward
        }
        return prev; // new head
    }
    public static void print(Node head) {
        while (head != null) {
            System.out.print(head.val + (head.next!=null?" -> ":"\n"));
            head = head.next;
        }
    }
    public static void main(String[] args) {
        Node head = new Node(1);
        head.next = new Node(2); head.next.next = new Node(3);
        print(reverse(head)); // 3 -> 2 -> 1
    }
}

```

6 Test Edge Cases

Test: empty list → null. Single node → same node. Two nodes.

7 Reflect & Optimize

Recursive approach exists but uses $O(n)$ stack. Iterative is $O(1)$ space and preferred.

Given a sorted array and a target, return the target's index using Binary Search.

Pattern

Binary Search (Divide & Conquer)

Input

arr={1,3,5,7,9,11,13},
target=7

Output

3

Complexity

Time: $O(\log n)$ | Space: $O(1)$

1 Read & Understand

Input: SORTED int array + target. Output: index or -1. Array must be sorted.

2 Work by Hand

mid=3 (val 7). $7==7 \rightarrow$ return 3. If not: $7>5 \rightarrow$ search right half; $7<9 \rightarrow$ search left half.

3 Identify Pattern

Divide search space in half each iteration. Eliminate half where target can't be.

4 Pseudocode

Pseudocode

```
SET low=0, high=arr.length-1
WHILE low <= high:
    mid = (low+high)/2
    IF arr[mid]==target: RETURN mid
    IF arr[mid]<target: low=mid+1
    ELSE: high=mid-1
RETURN -1
```

5 Write Java Code

Use $(low+high)/2$ — note: for very large arrays use $low+(high-low)/2$ to prevent overflow.

```
public class Main {  
    public static int binarySearch(int[] arr, int target) {  
        int low = 0, high = arr.length - 1;  
        while (low <= high) {  
            int mid = low + (high - low) / 2; // overflow-safe  
            if (arr[mid] == target) {  
                return mid;  
            } else if (arr[mid] < target) {  
                low = mid + 1; // search right  
            } else {  
                high = mid - 1; // search left  
            }  
        }  
        return -1;  
    }  
    public static void main(String[] args) {  
        int[] arr = {1,3,5,7,9,11,13};  
        System.out.println(binarySearch(arr, 7)); // 3  
        System.out.println(binarySearch(arr, 4)); // -1  
    }  
}
```

6 Test Edge Cases

Test: target at start, end, middle. Target not present. Array of one element.

7 Reflect & Optimize

Each iteration halves the search space → $O(\log n)$. 1 billion elements needs only ~30 comparisons.

Given two strings, check if one is an anagram of the other (same characters, different order).

Pattern
Frequency Array / HashMap

Input
"listen", "silent"

Output
true

Complexity
Time: $O(n)$ | Space: $O(1)$ with char array

1 Read & Understand

Input: two strings. Output: boolean. Anagram = same characters same frequency. Ignore case.

2 Work by Hand

"listen": l-1,i-1,s-1,t-1,e-1,n-1. "silent": same counts. Match $\rightarrow$ true.

3 Identify Pattern

Count frequency of each character in string1, then subtract for string2. If all zeros $\rightarrow$ anagram.

4 Pseudocode

Pseudocode

```
IF lengths differ: RETURN false
CREATE int[26] count
FOR each char in s1: count[c-a]++
FOR each char in s2: count[c-a]--
IF any count != 0: RETURN false
RETURN true
```

5 Write Java Code

int[26] array (for lowercase a-z) is $O(1)$ space — faster than HashMap.

Java — Complete Solution

```
public class Main {
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        int[] count = new int[26]; // for a-z
        for (char c : s1.toLowerCase().toCharArray()) count[c - 'a']++;
        for (char c : s2.toLowerCase().toCharArray()) count[c - 'a']--;
        for (int n : count) {
            if (n != 0) return false; // mismatch
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent")); // true
        System.out.println(isAnagram("hello", "world")); // false
    }
}
```

6 Test Edge Cases

Test: 'a','a'→true. 'a','b'→false. Different lengths→false. Same word→true.

7 Reflect & Optimize

Sorting both strings and comparing is $O(n \log n)$ — frequency array is $O(n)$.

Given an integer array, find the contiguous subarray with the largest sum.

Pattern

Dynamic Programming /
Greedy (Kadane's)

Input

nums={-2,1,-3,4,-1,2,1,-5,4}

Output

6 (subarray [4,-1,2,1])

Complexity

Time: O(n) | Space: O(1)

1 Read & Understand

Input: int array with negatives. Output: max sum of any contiguous subarray.

2 Work by Hand

Walk through: start with -2. At 1: $\max(1, -2+1)=1$. At -3: $\max(-3, 1-3)=-2$. At 4: $\max(4, -2+4)=4$. At -1: 3. At 2: 5. At 1: 6. Peak=6.

3 Identify Pattern

Kadane's: at each position, decide whether to extend previous subarray or start fresh. Track global max.

4 Pseudocode

Pseudocode

```
SET currMax = nums[0], globalMax = nums[0]
FOR i = 1 to n-1:
    currMax = max(nums[i], currMax+nums[i])
    globalMax = max(globalMax, currMax)
RETURN globalMax
```

5 Write Java Code

Math.max() for both comparisons. Initialize with first element to handle all-negative arrays.

Java — Complete Solution

```
public class Main {
    public static int maxSubarraySum(int[] nums) {
        int currMax = nums[0];
        int globalMax = nums[0];
        for (int i = 1; i < nums.length; i++) {
            // extend or start fresh?
            currMax = Math.max(nums[i], currMax + nums[i]);
            globalMax = Math.max(globalMax, currMax);
        }
        return globalMax;
    }
    public static void main(String[] args) {
        System.out.println(maxSubarraySum(new int[]{-2,1,-3,4,-1,2,1,-5,4}));
        // 6
        System.out.println(maxSubarraySum(new int[]{-3,-1,-2}));
        // -1 (all negative)
    }
}
```

6 Test Edge Cases

Test: all negative → return largest (least negative). All positive → sum of all. Single element.

7 Reflect & Optimize

$O(n)$ one-pass. No extra space. Kadane's is the standard solution — memorize it.

Given two sorted arrays, merge them into a single sorted array without using sort().

Pattern	Input	Output	Complexity
Two Pointers (Merge)	$a=\{1,3,5,7\}$, $b=\{2,4,6,8\}$	$\{1,2,3,4,5,6,7,8\}$	Time: $O(n+m)$ Space: $O(n+m)$

1 Read & Understand

Input: two sorted int arrays. Output: one merged sorted array. Sizes may differ.

2 Work by Hand

$i=0, j=0$: $a[0]=1 < b[0]=2 \rightarrow$ take 1. $i=1$: $a[1]=3 > b[0]=2 \rightarrow$ take 2. $j=1$: $a[1]=3 < b[1]=4 \rightarrow$ take 3... continue.

3 Identify Pattern

Two pointers, one per array. Compare current elements, take smaller, advance that pointer.

4 Pseudocode

Pseudocode

```
SET i=0, j=0, k=0
CREATE result[a.len + b.len]
WHILE i<a.len AND j<b.len:
    IF a[i]<=b[j]: result[k++]=a[i++]
    ELSE: result[k++]=b[j++]
COPY remaining elements
RETURN result
```

5 Write Java Code

After main loop, one array may have leftovers. Copy them using while loops.

```
import java.util.Arrays;

public class Main {
    public static int[] mergeSorted(int[] a, int[] b) {
        int[] result = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) {
            if (a[i] <= b[j]) result[k++] = a[i++];
            else result[k++] = b[j++];
        }
        while (i < a.length) result[k++] = a[i++]; // leftover a
        while (j < b.length) result[k++] = b[j++]; // leftover b
        return result;
    }
    public static void main(String[] args) {
        int[] merged = mergeSorted(new int[]{1,3,5,7}, new int[]{2,4,6,8});
        System.out.println(Arrays.toString(merged));
        // [1, 2, 3, 4, 5, 6, 7, 8]
    }
}
```

6 Test Edge Cases

Test: one empty array. Arrays of different sizes. Overlapping values.

7 Reflect & Optimize

This is the core of Merge Sort. Understanding this merge step unlocks merge sort.

Find the longest common prefix string among an array of strings.

Pattern

Vertical Scanning

Input

strs={"flower", "flow", "flight"}

Output

"fl"

ComplexityTime: $O(S)$ S =total chars |Space: $O(1)$ **1 Read & Understand**

Input: String array. Output: common prefix string or empty string. All strings may differ.

2 Work by Hand

"flower", "flow", "flight": col 0: f,f,f ✓. col 1: l,l,l ✓. col 2: o,l,i ✗ → stop. Prefix = "fl".

3 Identify Pattern

Vertical scan: compare character at same position across all strings. Stop at first mismatch.

4 Pseudocode*Pseudocode*

```
FOR each char position i in strs[0]:
    FOR each string s in strs:
        IF i>=s.length OR s.charAt(i)!=strs[0].charAt(i):
            RETURN strs[0].substring(0, i)
RETURN strs[0]
```

5 Write Java Code

Use first string as reference. Inner loop compares same position across all strings.

```
public class Main {  
    public static String longestPrefix(String[] strs) {  
        if (strs == null || strs.length == 0) return "";  
        for (int i = 0; i < strs[0].length(); i++) {  
            char c = strs[0].charAt(i); // reference char  
            for (String s : strs) {  
                if (i >= s.length() || s.charAt(i) != c) {  
                    return strs[0].substring(0, i);  
                }  
            }  
        }  
        return strs[0];  
    }  
    public static void main(String[] args) {  
        System.out.println(longestPrefix(new String[]{"flower", "flow", "flight"}));  
        // fl  
        System.out.println(longestPrefix(new String[]{"dog", "racecar", "car"}));  
        // (empty string)  
    }  
}
```

6 Test Edge Cases

Test: empty array → ". All same → full string. No common prefix → ". One string → that string.

7 Reflect & Optimize

Horizontal scan: fold with prefix, trim each step. Same complexity, different approach.

Check if a string is a palindrome considering only alphanumeric characters, ignoring cases.

Pattern Two Pointers + Filtering	Input "A man, a plan, a canal: Panama"	Output true	Complexity Time: O(n) Space: O(1)
-------------------------------------	--	----------------	--

1 Read & Understand

Input: String with spaces/punctuation. Output: boolean. Clean string first, then check palindrome.

2 Work by Hand

"A man a plan a canal Panama" → "amanaplanacanalpanama" → check palindrome → true.

3 Identify Pattern

Two pointers. Skip non-alphanumeric. Compare lowercase versions of valid chars.

4 Pseudocode

Pseudocode

```
SET left=0, right=s.length()-1
WHILE left < right:
    SKIP non-alphanumeric on both sides
    IF toLower(s[left]) != toLower(s[right]): RETURN false
    left++, right--
RETURN true
```

5 Write Java Code

Character.isLetterOrDigit() and Character.toLowerCase() are the key Java methods.

```
public class Main {  
    public static boolean isPalindrome(String s) {  
        int left = 0, right = s.length() - 1;  
        while (left < right) {  
            // skip non-alphanumeric  
            while (left < right && !Character.isLetterOrDigit(s.charAt(left)))  
                left++;  
            while (left < right && !Character.isLetterOrDigit(s.charAt(right)))  
                right--;  
            if (Character.toLowerCase(s.charAt(left)) !=  
                Character.toLowerCase(s.charAt(right)))  
                return false;  
            left++; right--;  
        }  
        return true;  
    }  
    public static void main(String[] args) {  
        System.out.println(isPalindrome("A man, a plan, a canal: Panama"));  
        // true  
        System.out.println(isPalindrome("race a car")); // false  
    }  
}
```

6 Test Edge Cases

Test: empty → true. All spaces → true. 'race a car' → false.

7 Reflect & Optimize

O(1) space — no cleaned string created. Directly skip bad chars with while loops inside main while.

★★★ HARD PROBLEMS (Problems 21–30)

Advanced algorithmic thinking: Dynamic Programming, Graph traversal, Backtracking, Binary Search on answer.

21

Longest Substring Without Repeating Characters

HARD

Find the length of the longest substring without repeating characters.

Pattern

Sliding Window + HashSet

Input

"abcabcbb"

Output

3 (abc)

Complexity

Time: $O(n)$ | Space:

$O(\min(n,m))$ $m=\text{charset}$

1 Read & Understand

Input: String. Output: integer (max length). A substring is contiguous. Characters are unique within window.

2 Work by Hand

"abcabcbb": window [a]=1, [ab]=2, [abc]=3. Next a repeats → shrink from left until a removed. Window [bca]=3... max=3.

3 Identify Pattern

Sliding window: expand right pointer. When duplicate found, shrink left pointer until duplicate removed.

4 Pseudocode

Pseudocode

```
SET left=0, maxlen=0
CREATE HashSet window
FOR right = 0 to n-1:
    WHILE s[right] in window: remove s[left], left++
    ADD s[right] to window
    maxlen = max(maxlen, right-left+1)
RETURN maxlen
```

5 Write Java Code

HashSet for $O(1)$ add/remove/contains. Window size = right - left + 1.

```
import java.util.HashSet;

public class Main {
    public static int lengthOfLongest(String s) {
        HashSet<Character> window = new HashSet<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            // shrink window until no duplicate
            while (window.contains(s.charAt(right))) {
                window.remove(s.charAt(left));
                left++;
            }
            window.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }

    public static void main(String[] args) {
        System.out.println(lengthOfLongest("abcabcbb")); // 3
        System.out.println(lengthOfLongest("bbbbbb")); // 1
        System.out.println(lengthOfLongest("pwwkew")); // 3
    }
}
```

6 Test Edge Cases

Test: all same 'aaa'→1. All unique 'abc'→3. Empty→0. Single char→1.

7 Reflect & Optimize

HashMap approach: store char→index, jump left directly. Faster in practice, same complexity.

Given an elevation map (array of heights), compute how much water it can trap after raining.

Pattern

Two Pointers (Optimal) or
Precompute max arrays

Input

height={0,1,0,2,1,0,1,3,2,1,2,
1}

Output

6

Complexity

Time: $O(n)$ | Space: $O(1)$

1 Read & Understand

Input: int array of heights. Output: total water units trapped. Water above a column = $\min(\text{leftMax}, \text{rightMax}) - \text{height}$.

2 Work by Hand

Water at each index = $\min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$. Index 2: $\min(1, 3) - 0 = 1$. Index 5: $\min(2, 3) - 0 = 2$. Sum all.

3 Identify Pattern

Two-pointer approach: left and right pointers. Track leftMax and rightMax. Process the side with smaller max.

4 Pseudocode

Pseudocode

```
SET left=0, right=n-1, leftMax=0, rightMax=0, water=0
WHILE left <= right:
    IF height[left] <= height[right]:
        if h[left]>=leftMax: leftMax=h[left], else water+=leftMax-h[left]; left++
    ELSE: same logic on right side
RETURN water
```

5 Write Java Code

The key insight: if $\text{leftMax} < \text{rightMax}$, water at left is determined by leftMax. Process that side.

```
public class Main {  
    public static int trap(int[] h) {  
        int left = 0, right = h.length - 1;  
        int leftMax = 0, rightMax = 0, water = 0;  
        while (left <= right) {  
            if (h[left] <= h[right]) {  
                if (h[left] >= leftMax) leftMax = h[left];  
                else water += leftMax - h[left];  
                left++;  
            } else {  
                if (h[right] >= rightMax) rightMax = h[right];  
                else water += rightMax - h[right];  
                right--;  
            }  
        }  
        return water;  
    }  
    public static void main(String[] args) {  
        System.out.println(trap(new int[]{0,1,0,2,1,0,1,3,2,1,2,1})); // 6  
        System.out.println(trap(new int[]{4,2,0,3,2,5})); // 9  
    }  
}
```

6 Test Edge Cases

Test: flat array→0. Monotone increasing→0. V-shape: {3,0,3}→3.

7 Reflect & Optimize

This is a classic hard problem. Two-pointer is $O(1)$ space vs $O(n)$ for precomputed arrays.

Given a 2D board of characters and a word, return true if the word exists as adjacent (horizontal/vertical) cells.

Pattern DFS + Backtracking	Input board={{A,B,C,E},{S,F,C,S},{A,D,E,E}}, word='ABCCED'	Output true	Complexity Time: $O(m*n*4^L)$ L=word length Space: $O(L)$
-------------------------------	---	----------------	--

1 Read & Understand

Input: `char[][] board`, `String word`. Output: `boolean`. Each cell can only be used once per path.

2 Work by Hand

Find 'A' at (0,0). Try right→'B' found. Try right→'C' found. Try down→'C' found. Try left→'E' found. Try down→'D' found. Word complete!

3 Identify Pattern

DFS from every cell. At each step, mark cell visited. Try all 4 directions. If word complete → true. Backtrack if stuck.

4 Pseudocode

Pseudocode

```
FOR each cell (r,c):
    IF dfs(r,c,0) == true: RETURN true
FUNCTION dfs(r,c,idx):
    IF idx==word.length: RETURN true
    IF out-of-bounds OR visited OR board[r][c]!=word[idx]: RETURN false
    MARK visited (board[r][c]='#')
    TRY all 4 directions
    UNMARK (backtrack)
    RETURN result
```

5 Write Java Code

Mark cell with '#' to avoid revisiting. Restore after backtrack. Critical!

```

public class Main {
    static int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    public static boolean exist(char[][] board, String word) {
        int m = board.length, n = board[0].length;
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++)
                if (dfs(board, word, r, c, 0)) return true;
        return false;
    }
    static boolean dfs(char[][] b, String w, int r, int c, int idx) {
        if (idx == w.length()) return true;
        if (r < 0 || r >= b.length || c < 0 || c >= b[0].length) return false;
        if (b[r][c] != w.charAt(idx)) return false;
        char temp = b[r][c];
        b[r][c] = '#'; // mark visited
        for (int[] d : dirs)
            if (dfs(b, w, r+d[0], c+d[1], idx+1)) {
                b[r][c] = temp; return true;
            }
        b[r][c] = temp; // backtrack
        return false;
    }
    public static void main(String[] args) {
        char[][] board = {{'A','B','C','E'},{'S','F','C','S'},{'A','D','E','E'}};
        System.out.println(exist(board, "ABCCED")); // true
    }
}

```

6 Test Edge Cases

Test: word longer than board. Word not present. Word at corners. Single cell board.

7 Reflect & Optimize

Pruning: if first character doesn't match, skip. Can precount character frequencies to fail fast.

Merge k sorted linked lists into one sorted list.

Pattern

Min-Heap (Priority Queue)

Input

lists=[[1,4,5],[1,3,4],[2,6]]

Output

[1,1,2,3,4,4,5,6]

Complexity

Time: $O(N \log k)$ N =total nodes | Space: $O(k)$

1 Read & Understand

Input: k sorted LinkedLists. Output: one merged sorted LinkedList. Cannot sort at end — must use structure.

2 Work by Hand

Add head of each list to min-heap. Extract min (1), add to result. Push that node's next to heap. Repeat.

3 Identify Pattern

Min-Heap maintains the smallest element across all k lists at $O(\log k)$ per operation.

4 Pseudocode

Pseudocode

```
ADD head of each list to PriorityQueue
SET dummy = new Node(0), curr = dummy
WHILE heap not empty:
    node = heap.poll() (smallest)
    curr.next = node, curr = curr.next
    IF node.next != null: heap.add(node.next)
RETURN dummy.next
```

5 Write Java Code

PriorityQueue with a Comparator on node values. Dummy head simplifies result construction.

```

import java.util.PriorityQueue;
public class Main {
    static class Node { int val; Node next; Node(int v){val=v;} }
    public static Node mergeKLists(Node[] lists) {
        PriorityQueue<Node> heap = new PriorityQueue<>((a,b)->a.val-b.val);
        for (Node head : lists)
            if (head != null) heap.offer(head);
        Node dummy = new Node(0), curr = dummy;
        while (!heap.isEmpty()) {
            Node node = heap.poll(); // smallest
            curr.next = node;
            curr = curr.next;
            if (node.next != null) heap.offer(node.next);
        }
        return dummy.next;
    }
    public static void main(String[] args) {
        // Build lists [1->4->5], [1->3->4], [2->6]
        Node l1=new Node(1); l1.next=new Node(4); l1.next.next=new Node(5);
        Node l2=new Node(1); l2.next=new Node(3); l2.next.next=new Node(4);
        Node l3=new Node(2); l3.next=new Node(6);
        Node res=mergeKLists(new Node[]{l1,l2,l3});
        while(res!=null){System.out.print(res.val+" ");res=res.next;}
        // 1 1 2 3 4 4 5 6
    }
}

```

6 Test Edge Cases

Test: empty lists. One list. All lists have one element. Lists of different sizes.

7 Reflect & Optimize

Alternative: merge two lists at a time $\rightarrow O(kN)$. Divide and conquer: $O(N \log k)$. Heap is also $O(N \log k)$ but simpler.

25

Coin Change (Minimum Coins)

HARD

Given coins of different denominations and an amount, return the fewest number of coins to make that amount.

Pattern

Dynamic Programming
(Bottom-Up)

Input

coins={1,5,6,9}, amount=11

Output

2 (5+6)

Complexity

Time: $O(\text{amount} * \text{coins})$ |

Space: $O(\text{amount})$

1 Read & Understand

Input: `int[] coins` + target amount. Output: minimum coin count or -1 if impossible.

2 Work by Hand

Greedy fails here: 11 with {1,5,6,9}: greedy picks $9+1+1=3$ coins. DP finds $5+6=2$ coins.

3 Identify Pattern

DP table: `dp[i]` = min coins to make amount `i`. For each amount, try all coins.

4 Pseudocode

Pseudocode

```
CREATE dp[amount+1] filled with amount+1 (infinity)
dp[0] = 0
FOR i = 1 to amount:
    FOR each coin:
        IF coin <= i:
            dp[i] = min(dp[i], dp[i-coin]+1)
RETURN dp[amount] > amount ? -1 : dp[amount]
```

5 Write Java Code

Initialize `dp` with `amount+1` as 'infinity' (impossible). `dp[0]=0` is the base case.

```
import java.util.Arrays;

public class Main {
    public static int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1); // 'infinity'
        dp[0] = 0; // base case
        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }

    public static void main(String[] args) {
        System.out.println(coinChange(new int[]{1,5,6,9}, 11)); // 2
        System.out.println(coinChange(new int[]{2}, 3));        // -1
        System.out.println(coinChange(new int[]{1,2,5}, 11));   // 3
    }
}
```

6 Test Edge Cases

Test: amount=0→0. No combination→-1. Single coin equals amount→1.

7 Reflect & Optimize

This is the classic DP pattern: define subproblem $dp[i]$, relate to smaller subproblems. Memorize this structure.

26

Longest Increasing Subsequence (LIS)

HARD

Find the length of the longest strictly increasing subsequence in an array.

Pattern

Dynamic Programming

Input

nums={10,9,2,5,3,7,101,18}

Output

4 (2,3,7,101 or 2,5,7,101)

Complexity

Time: $O(n^2)$ | Space: $O(n)$

1

Read & Understand

Input: int array. Output: length of LIS. Subsequence = elements in order but not necessarily contiguous.

2

Work by Hand

At index 6 (101): what's the longest increasing subsequence ending at 101? Check all previous smaller elements and pick max dp.

3

Identify Pattern

$dp[i]$ = length of LIS ending at index i . For each i , look back at all $j < i$ where $nums[j] < nums[i]$.

4

Pseudocode

Pseudocode

```
CREATE dp[n] filled with 1 (each element is LIS of length 1)
FOR i = 1 to n-1:
    FOR j = 0 to i-1:
        IF nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j]+1)
RETURN max(dp)
```

5

Write Java Code

Initialize all $dp[i]=1$. Track global max after each update.

```
public class Main {  
    public static int lis(int[] nums) {  
        int n = nums.length;  
        int[] dp = new int[n];  
        java.util.Arrays.fill(dp, 1); // each element alone = length 1  
        int maxLen = 1;  
        for (int i = 1; i < n; i++) {  
            for (int j = 0; j < i; j++) {  
                if (nums[j] < nums[i]) {  
                    dp[i] = Math.max(dp[i], dp[j] + 1);  
                }  
            }  
            maxLen = Math.max(maxLen, dp[i]);  
        }  
        return maxLen;  
    }  
    public static void main(String[] args) {  
        System.out.println(lis(new int[]{10,9,2,5,3,7,101,18})); // 4  
        System.out.println(lis(new int[]{0,1,0,3,2,3})); // 4  
    }  
}
```

6 Test Edge Cases

Test: already sorted→n. Reverse sorted→1. All same→1. Single element→1.

7 Reflect & Optimize

$O(n \log n)$ solution exists using Binary Search + patience sorting. Reduces from $O(n^2)$. Advanced.

Given a 2D grid of '1's (land) and '0's (water), count the number of islands.

Pattern BFS / DFS Graph Traversal	Input grid={{1,1,0,0},{1,1,0,0},{0,0,1,0},{0,0,0,1}}	Output 3	Complexity Time: $O(m*n)$ Space: $O(m*n)$
--------------------------------------	---	-------------	--

1 Read & Understand

Input: `char[][] grid`. Output: integer count. Island = connected group of '1's (4-directional).

2 Work by Hand

Find a '1' at (0,0). BFS/DFS marks all connected '1's as visited. Count=1. Find next unvisited '1'... count=2, then 3.

3 Identify Pattern

Iterate all cells. When unvisited '1' found, increment count and DFS to mark entire island.

4 Pseudocode

Pseudocode

```
SET count = 0
FOR each cell (r,c):
    IF grid[r][c]=='1':
        count++
        dfs(grid,r,c) to mark entire island
FUNCTION dfs: mark cell '0', recurse on 4 neighbors if '1'
RETURN count
```

5 Write Java Code

Modify grid in-place (mark '0') to avoid visited array. Simple but destructive.

```

public class Main {
    public static int numIslands(char[][] grid) {
        int count = 0;
        for (int r = 0; r < grid.length; r++) {
            for (int c = 0; c < grid[0].length; c++) {
                if (grid[r][c] == '1') {
                    count++;
                    dfs(grid, r, c); // sink the island
                }
            }
        }
        return count;
    }

    static void dfs(char[][] g, int r, int c) {
        if (r < 0 || r >= g.length || c < 0 || c >= g[0].length || g[r][c] != '1') return;
        g[r][c] = '0'; // mark visited
        dfs(g, r+1, c); dfs(g, r-1, c);
        dfs(g, r, c+1); dfs(g, r, c-1);
    }

    public static void main(String[] args) {
        char[][] grid = {{1,1,0,0},{1,1,0,0},{0,0,1,0},{0,0,0,1}};
        // convert ints to char for demo:
        char[][] g = {{'1','1','0','0'},{'1','1','0','0'},
                      {'0','0','1','0'},{'0','0','0','1'}};
        System.out.println(numIslands(g)); // 3
    }
}

```

6 Test Edge Cases

Test: all water→0. All land→1. Diagonal islands (not connected)→each cell is island.

7 Reflect & Optimize

BFS alternative: use a Queue. Same complexity. DFS uses recursion (stack). Both are valid.

Design an algorithm to serialize a binary tree to a string and deserialize that string back to the original tree.

Pattern DFS Pre-order Traversal + Queue Parsing	Input Tree: 1(2,3(4,5))	Output Serialize→'1,2,null,null,3,4,null,null,5,null,null' → Deserialize back to same tree	Complexity Time: O(n) Space: O(n)
--	-----------------------------------	---	---

1 Read & Understand

Input: TreeNode root (serialize) / String (deserialize). Both must be consistent and reversible.

2 Work by Hand

Pre-order: root,left,right. Node 1→2→null→null→3→4→null→null→5→null→null. Use queue to rebuild.

3 Identify Pattern

Serialize: pre-order traversal, write 'null' for missing nodes. Deserialize: consume tokens from queue using same pre-order.

4 Pseudocode

Pseudocode

```
SERIALIZE: DFS preorder, append val or 'null' with comma separator
DESERIALIZE: split by comma, create queue
    function build(): poll token, if null return null
    node=new Node(token)
    node.left=build(), node.right=build()
    return node
```

5 Write Java Code

Use StringBuilder for efficient string building. Use ArrayDeque as queue for token consumption.

```

import java.util.*;
public class Main {
    static class TreeNode { int val; TreeNode left, right; TreeNode(int v){val=v;} }
    public static String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        serDFS(root, sb);
        return sb.toString();
    }
    static void serDFS(TreeNode node, StringBuilder sb) {
        if (node == null) { sb.append("null,"); return; }
        sb.append(node.val).append(',');
        serDFS(node.left, sb);
        serDFS(node.right, sb);
    }
    public static TreeNode deserialize(String data) {
        Queue<String> q = new ArrayDeque<>(Arrays.asList(data.split(",")));
        return desDFS(q);
    }
    static TreeNode desDFS(Queue<String> q) {
        String val = q.poll();
        if (val.equals("null")) return null;
        TreeNode node = new TreeNode(Integer.parseInt(val));
        node.left = desDFS(q);
        node.right = desDFS(q);
        return node;
    }
    public static void main(String[] args) {
        TreeNode root = new TreeNode(1);
        root.left=new TreeNode(2); root.right=new TreeNode(3);
        root.right.left=new TreeNode(4); root.right.right=new TreeNode(5);
        String s = serialize(root);
        System.out.println(s);
        TreeNode restored = deserialize(s);
        System.out.println(serialize(restored)); // same string
    }
}

```

6 Test Edge Cases

Test: null root. Single node. Complete binary tree. Left-skewed tree.

7 Reflect & Optimize

Level-order (BFS) serialization is also valid and produces shorter strings. Pre-order is simpler to implement.

Find the median of two sorted arrays in $O(\log(\min(m,n)))$ time.

Pattern

Binary Search on Partition

Input

nums1={1,3}, nums2={2}

Output

2.0

ComplexityTime: $O(\log(\min(m,n)))$ |Space: $O(1)$ **1 Read & Understand**

Input: two sorted int arrays. Output: double (median). Combined array size may be odd or even.

2 Work by Hand

Combined {1,2,3}: median=2. For even {1,2,3,4}: median=(2+3)/2=2.5. Use binary search on smaller array partition.

3 Identify Pattern

Partition both arrays such that left halves contain $(m+n)/2$ elements. Use BS on smaller array. Verify partition is valid.

4 Pseudocode*Pseudocode*

```
ENSURE nums1 is shorter (swap if needed)
SET lo=0, hi=m
WHILE lo<=hi:
    i=partition in nums1, j=(m+n+1)/2-i
    IF nums1[i-1]>nums2[j]: hi=i-1
    ELIF nums2[j-1]>nums1[i]: lo=i+1
    ELSE: found partition, calc median
```

5 Write Java Code

Handle edge cases with Integer.MIN/MAX_VALUE for boundaries. Odd/even combined length changes median formula.

```

public class Main {
    public static double findMedian(int[] a, int[] b) {
        if (a.length > b.length) return findMedian(b, a); // a is shorter
        int m=a.length, n=b.length;
        int lo=0, hi=m;
        while (lo <= hi) {
            int i = (lo+hi)/2, j = (m+n+1)/2 - i;
            int aLeft = (i==0) ? Integer.MIN_VALUE : a[i-1];
            int aRight = (i==m) ? Integer.MAX_VALUE : a[i];
            int bLeft = (j==0) ? Integer.MIN_VALUE : b[j-1];
            int bRight = (j==n) ? Integer.MAX_VALUE : b[j];
            if (aLeft <= bRight && bLeft <= aRight) {
                if ((m+n)%2==1) return Math.max(aLeft,bLeft);
                return (Math.max(aLeft,bLeft)+Math.min(aRight,bRight))/2.0;
            } else if (aLeft > bRight) hi = i-1;
            else lo = i+1;
        }
        return 0.0;
    }
    public static void main(String[] args) {
        System.out.println(findMedian(new int[]{1,3}, new int[]{2})); // 2.0
        System.out.println(findMedian(new int[]{1,2}, new int[]{3,4})); // 2.5
    }
}

```

6 Test Edge Cases

Test: one empty array. Arrays of same size. All elements of one array smaller than other.

7 Reflect & Optimize

This is one of the hardest binary search problems. Understanding the partition invariant is key.

Given a sorted list of words in an alien language, find the order of characters.

Pattern

Graph + Topological Sort
(BFS/Kahn's Algorithm)

Input

words={wrt,wrf,er,ett,rftt}

Output

wertf

Complexity

Time: $O(C)$ C =total chars |
Space: $O(1)$ (26 chars max)

1 Read & Understand

Input: `String[] words` sorted in alien order. Output: character ordering string. May have multiple valid answers.

2 Work by Hand

Compare adjacent words: wrt vs wrf $\rightarrow$ t comes before f. wrf vs er $\rightarrow$ w comes before e. er vs ett $\rightarrow$ r before t. Build directed graph edges.

3 Identify Pattern

Build directed graph from adjacent word comparisons. Then topological sort (Kahn's BFS) on the graph.

4 Pseudocode*Pseudocode*

```
BUILD graph: compare adjacent word pairs, first differing char  $\rightarrow$  edge
COMPUTE in-degrees for all chars
BFS (Kahn's): enqueue chars with in-degree 0
    Process: reduce neighbor in-degrees, enqueue zeros
IF result length < unique chars: cycle (invalid)
RETURN result string
```

5 Write Java Code

Use List< for adjacency. Handle edge case: longer word before prefix $\rightarrow$ return "".

```

import java.util.*;
public class Main {
    public static String alienOrder(String[] words) {
        Map<Character, Set<Character>> adj = new HashMap<>();
        Map<Character, Integer> inDeg = new HashMap<>();
        for (String w : words) for (char c : w.toCharArray()) {
            adj.putIfAbsent(c, new HashSet<>());
            inDeg.putIfAbsent(c, 0);
        }
        for (int i=0; i<words.length-1; i++) {
            String w1=words[i], w2=words[i+1];
            int len=Math.min(w1.length(),w2.length()); boolean found=false;
            for (int j=0; j<len; j++) {
                if (w1.charAt(j)!=w2.charAt(j)) {
                    if (!adj.get(w1.charAt(j)).contains(w2.charAt(j))) {
                        adj.get(w1.charAt(j)).add(w2.charAt(j));
                        inDeg.merge(w2.charAt(j), 1, Integer::sum);
                    }
                    found=true; break;
                }
            }
            if (!found && w1.length()>w2.length()) return "";
        }
        Queue<Character> q = new ArrayDeque<>();
        for (char c : inDeg.keySet()) if (inDeg.get(c)==0) q.offer(c);
        StringBuilder res = new StringBuilder();
        while (!q.isEmpty()) {
            char c = q.poll(); res.append(c);
            for (char nb : adj.get(c)) {
                inDeg.merge(nb, -1, Integer::sum);
                if (inDeg.get(nb)==0) q.offer(nb);
            }
        }
        return res.length()==inDeg.size() ? res.toString() : "";
    }
    public static void main(String[] args) {
        System.out.println(alienOrder(new String[]{"wrt","wrf","er","ett","rftt"}));
        // wertf (one valid order)
    }
}

```

6 Test Edge Cases

Test: single word. Two words same character. Invalid (cycle) → return "".

7 Reflect & Optimize

This combines graph construction + BFS topological sort. A very common advanced interview problem.

COMPLETE PROBLEM SUMMARY — ALL 30 PROBLEMS

#	TITLE	LEVEL	PATTERN	COMPLEXITY
1	Print Numbers 1 to N	BASIC	Loop / Counting	Time: O(n)
2	Check Even or Odd	BASIC	Condition / Modulo	Time: O(1)
3	Sum of Digits of a Number	BASIC	Digit Extraction with % and /	Time: O(d) where d = digits
4	Check Prime Number	BASIC	Loop + Early Exit Optimization	Time: O(sqrt(n))
5	Factorial of a Number	BASIC	Accumulator / Iterative Multiplication	Time: O(n)
6	Reverse a String	BASIC	Two Pointers	Time: O(n)
7	Count Vowels in a String	BASIC	Linear Scan + Condition	Time: O(n)
8	Find GCD of Two Numbers	BASIC	Euclidean Algorithm	Time: O(log(min(a,b)))
9	Linear Search in Array	BASIC	Linear Search	Time: O(n)
10	Multiplication Table	BASIC	Loop / Formatted Output	Time: O(1)
11	Two Sum — Find Pair with Target	INTERMEDIATE	HashMap for O(n) lookup	Time: O(n)
12	Check Balanced Parentheses	INTERMEDIATE	Stack (LIFO)	Time: O(n)
13	Find All Duplicates in Array	INTERMEDIATE	Frequency Map (HashMap)	Time: O(n)
14	Reverse a Linked List	INTERMEDIATE	Three-Pointer Reversal (Iterative)	Time: O(n)
15	Binary Search	INTERMEDIATE	Binary Search (Divide & Conquer)	Time: O(log n)
16	Anagram Check	INTERMEDIATE	Frequency Array / HashMap	Time: O(n)
17	Maximum Subarray Sum (Kadane's Algorithm)	INTERMEDIATE	Dynamic Programming / Greedy (Kadane's)	Time: O(n)
18	Merge Two Sorted Arrays	INTERMEDIATE	Two Pointers (Merge)	Time: O(n+m)
19	Longest Common Prefix	INTERMEDIATE	Vertical Scanning	Time: O(S) S=total chars
20	Valid Palindrome (Alphanumeric Only)	INTERMEDIATE	Two Pointers + Filtering	Time: O(n)
21	Longest Substring Without Repeating Characters	HARD	Sliding Window + HashSet	Time: O(n)
22	Trapping Rain Water	HARD	Two Pointers (Optimal) or Precompute Max Arrays	Time: O(n)
23	Word Search in 2D Grid (DFS Backtracking)	HARD	DFS + Backtracking	Time: O(m*n*4^L) L=word length
24	Merge K Sorted Lists	HARD	Min-Heap (Priority Queue)	Time: O(N log k) N=total nodes
25	Coin Change (Minimum Coins)	HARD	Dynamic Programming (Bottom-Up)	Time: O(amount * coins)
26	Longest Increasing Subsequence (LIS)	HARD	Dynamic Programming	Time: O(n^2)
27	Number of Islands (BFS/DFS on Grid)	HARD	BFS / DFS Graph Traversal	Time: O(m*n)
28	Serialize and Deserialize Binary Tree	HARD	DFS Pre-order Traversal + Queue Parsing	Time: O(n)
29	Median of Two Sorted Arrays	HARD	Binary Search on Partition	Time: O(log(min(m,n)))
30	Alien Dictionary (Topological Sort)	HARD	Graph + Topological Sort (BFS/Kahn's)	Time: O(C) C=total chars

The 7-Step Framework Works. Use It Every Time.

Read the problem twice. ● Solve by hand first. ● Name the pattern. Write pseudocode. ● Code it in Java. ●
Test edge cases. ● Optimize.

